

Given an array of positive integers, Can we find a subset of the array such that the sum of the subset equals to a target x ?

e.g. $[1, 5, 11, 5]$, target = 11 $\rightarrow$ Subset: $[1, 5, 5]$

Solution 1: brute force

def recursive(index, target):

if target == 0:

return True

if target < 0 or index >= len(nums):

return False

return recursive(index, target - nums[index]) ^{$\nearrow$ include index} or recursive(index + 1, target)

$\searrow$ exclude index

return recursive(0, x)

time complexity: $O(2^n)$, where $n = \text{len}(\text{nums})$,

$\uparrow$
We consider all subsets of the array, until find the True case.

However, since we have $0 \leq \text{target} \leq \text{Sum}(\text{nums}) = m$

$$0 \leq \text{index} \leq n-1$$

We know there are at most $n \times m$ different states!

Why is this? $\rightarrow$ Simply use recursion compute the same sub-function multiple times

$\Downarrow$
Consider Fibonacci.

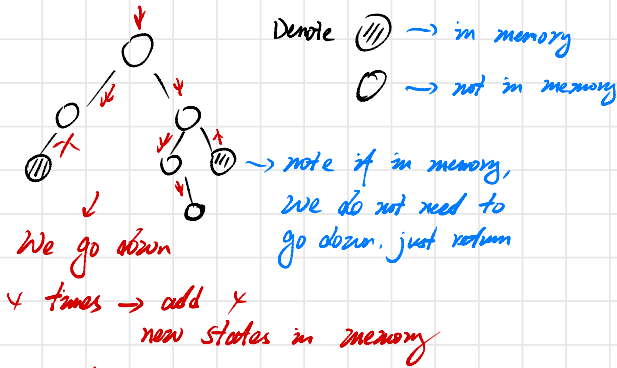
How to solve this in recursion?

Solution 1: Simply add @ cache before the recursive function

↓
reduce time complexity to $O(mn)$

→ each distinct state requires $O(1)$

Example (See ^{also} next page)



★
→ how many times can we go down?
→ at most $m \cdot n$ times

→ each time we go down: met a new state not in memory
and add it into memory

★
→ how many times can we go up?

bounded by $2 \times$ times go down

Solution 2: dynamic programming

{ 1, 3 } → all possible subset sum

↓
{ 1, 3, 2 } → update all possible subset sum

order to memory: $1 \rightarrow 2 \rightarrow 3 \dots$

each one take only

constant time

Recursive tree for nums = [6,7,4,3] and subSetSum = 10

